

第 1 章 静的親データの生成 — N=3..40 自己無撞着円偏波固有モード親と零閉塞核種

木原 範昭 (WF System Co., Ltd.) ORCID 0009-0004-6753-4020

2026 年 9 月 5 日 Version DOI 10.5281/zenodo.22317636

(N=3..40 段 1+2+3 スイープ再現論文・第 1 章。総括論文と Concept DOI 10.5281/zenodo.22317635 を共有する。Version DOI 10.5281/zenodo.22317636)

1. 目的

本章は、第 3 章のスイープ (N=3..40、段 1+2+3 力学) が入力とする初期データ—— 自己無撞着円偏波固有モード親 v 、零閉塞核種 g 、および正規化初期状態 Z_0 ——を、乱数種表式から決定的に生成し、静的ファイル (npz) として固定する数値実験を記述する。

生成手順・乱数消費順・全引数は 2026-07-22 の正本走行自発的分裂予備実験_v1/run_spontaneous_splitting_largeN_v1.py (N=40, $\delta=1e-15$, seed=0, tol=1e-12) と同一であり、N=40 について生成物が正本走行の初期値と bit 一致することを合格ゲートとする。本章の目的は完全な再現性と数式・プログラム・データの対応の固定であり、物理的解釈は含まない。

2. 理論的背景

完全グラフ K_N の辺集合上の複素関係波を扱う。

(式 1) 辺集合と次元 — 辺数 $M = N(N - 1)/2$ 。辺の順序は上三角順 $((0,1),(0,2),\dots,(0,N - 1),(1,2),\dots)$ で固定する。状態は $z \in \mathbb{C}^M$ 。

(式 2) 位相差正弦生成子 — 辺位相 $\theta \in \mathbb{R}^M$ に対し、頂点を共有する辺対 (e,f) 上で

$$K_{ef}(\theta) = \cos \theta_e \sin \theta_f - \sin \theta_e \cos \theta_f = \sin(\theta_f - \theta_e)$$

非隣接では 0、対角 0。K は実反対称 ($K^T = -K$)。

(式 3) 頂点分解と小空間 — $c_e = \cos \theta_e$, $s_e = \sin \theta_e$ を頂点へ散布した行列 C, S ($M \times N$) を並べ $W = [C|S]$ ($M \times 2N$)、 $J = \begin{bmatrix} 0 & I_N \\ -I_N & 0 \end{bmatrix}$ とすると

$$K = C S^T - S C^T = W J W^T, \quad \text{rank } K \leq 2N$$

グラム行列 $G = W^T W$ ($2N \times 2N$) は、 K_N の線グラフでは頂点对 (k,l) の共有辺が 1 本 (辺 (k,l)) であることから解析的に構成できる:

$$G_{cc}[k,l] = \cos^2 \theta_{\{(k,l)\}}, \quad G_{cs}[k,l] = \cos \theta_{\{(k,l)\}} \sin \theta_{\{(k,l)\}}, \quad G_{ss}[k,l] = \sin^2 \theta_{\{(k,l)\}} \quad (k \neq l)$$

対角は各行の非対角和

(式 4) 円偏波固有モード (親) — K の非零スペクトルは JG ($2N \times 2N$) の固有値と対応する。 JG の固有値 λ のうち虚部最小のもの ($\lambda = -i \sigma_{\max}$) の固有ベクトル y から

$$v = W y, \quad v \leftarrow v / \|v\|$$

を作る。 iK はエルミートなので、 v が固有モードなら $iKv = \mu v$ ($\mu = \sigma_{\max}$)。

(式 5) 自己無撞着反復 (位相混合) — v から読み出した位相 $\theta_{\text{new}} = \arg v$ を混合率 $\beta = 0.5$ で現位相に混ぜる:

$$\theta \leftarrow \arg((1 - \beta) e^{i\theta} + \beta e^{i\theta_{\text{new}}})$$

この反復の固定点が「自分の位相から作った生成子の固有モードが自分自身」という自己無撞着親である。

(式 6) 固有モード残差 (収束判定量) —

$$\mu = \text{Re}(v^\dagger (iKv)), \quad r = \|iKv - \mu v\|$$

$r < \text{tol}$ (本章では $\text{tol} = 1e-12$) で収束と判定する。

(式 7) 零閉塞核種 — $\text{span}(W)$ の直交補への射影を

$$P \perp g = g - W q, \quad q = \text{lstsq}(G, W^T g)$$

とし、実ガウスベクトル ξ_1, ξ_2 から $u = P \perp \xi_1 / \|\cdot\|$ 、 w = 正規直交化した $P \perp \xi_2$ を作り

$$g = (u + i w) / \sqrt{2}$$

とする。 $u \perp w$ 、 $\|u\| = \|w\| = 1$ より $g^T g = (\|u\|^2 - \|w\|^2)/2 + i u \cdot w = 0$ が厳密に成り立つ (零閉塞)。また $g \in \text{span}(W)^\perp$ より g は K の値域の外にある。

(式 8) 初期状態 —

$$z_0 = (v + \delta g) / \|v + \delta g\|, \quad \delta = 1e-15$$

(式 9) 乱数 — 乱数生成器は numpy PCG64 であり、種は

$$\text{seed}(N) = 40260722 + 1000 \cdot N + 0$$

消費順は「親反復の初期位相 θ_0 (m 個の一樣乱数、リスタート毎) $\rightarrow \xi_1$ (m 個の正規乱数) $\rightarrow \xi_2$ (m 個の正規乱数)」で固定。これにより全生成物が N のみの決定的関数になる。

3. 実装方法

- エンジン `run_n_scaling_lowrank_v1.py` は 2026-07-22 正本(自発的分裂予備実験_v1/run_n_scaling_lowrank_v1. の bit 同一コピー (diff で確認)。式 2~式 7 の実装はすべてこのエンジンにあり、本章では**一切変更せず import で使用する** (独自再実装の禁止規約に従う)。
- 生成プログラム `make_static_parents_N3_N40_v1.py` は、正本走行 `run()` 冒頭と同一の呼び出し列 (式 9 の `rng` $\rightarrow$ `make_parent` $\rightarrow$ `zero_closure_kernel_seed` $\rightarrow$ 式 8 の正規化) を $N=3..40$ について実行し、npz に保存する。

- N=40 は正本静的親 (2026-09-04、正本走行との bit 一致検証済み) との bit 一致を プログラム内ゲートとして検証する。

4. 詳細設計

4.1 全体フロー

for N in 3..40:

[初期化] LowRankSystem(N) を構築 (辺順序・J)、rng = default_rng(40260722+1000N)

[ループ処理] make_parent: 自己無撞着反復 (式 4 → 式 5 → 式 6、最大 1200 反復 × 3 リスタート)

zero_closure_kernel_seed: 種 g の構成 (式 7)

[終了処理] Z0 の構成 (式 8) → N=40 ゲート → npz 保存 (既存なら検証のみ) → 台帳へ追記
台帳 parents_summary.csv を出力、ゲート不合格なら exit 1

4.2 全体データフロー

- 入力: ファイル入力なし。乱数種表式 (式 9) と定数のみ。
 - SEED = 0 ... 種表式の第 3 項 (系列番号)
 - DELTA = $1e-15$... 式 8 の δ (種振幅)
 - TOL = $1e-12$... 式 6 の収束閾値 (正本走行の --tol= $1e-12$ と同一)
 - ITERS = 1200 ... 自己無撞着反復の上限 (正本走行 run() の iters=1200 と同一)
 - 参照入力 REF40 ... N=40 ゲート用の正本静的親 npz (読み出しのみ)
- 処理: 4.3 の個別処理を N=3..40 で反復。
- 出力:
 - parents/parent_static_N{N:05d}_makeparent_20260905.npz × 38 (フィールド: v, g, Z0, sigma (親の σ スペクトル), residual, n, seed, delta, tol, iters)
 - parents/parents_summary.csv (列: N, M, parent_residual, rank_planes, status)

定数定義 (make_static_parents_N3_N40_v1.py) :

```

34 PARENT_DIR = os.path.join(BASE_DIR, "parents")
35 REF40 = '../自発的分裂予備実験_v1_N40 対照実験系_20260904/largeN_splitting_result_v1/parent_sta
36
37 SEED = 0
38 DELTA = 1e-15
39 TOL = 1e-12
40 ITERS = 1200

```

4.3 個別処理

■4.3.1 初期化 エンジン初期化 (式 1・式 3 の J)。辺順序は np.triu_indices で固定される。

run_n_scaling_lowrank_v1.py:

```

52 def build_edges(n):
53     ea, eb = np.triu_indices(n, k=1)
54     return ea.astype(np.int64), eb.astype(np.int64)
...

```

```

60     def __init__(self, n):
61         self.n = n
62         self.ea, self.eb = build_edges(n)
63         self.m = len(self.ea)
64         self.J = np.zeros((2 * n, 2 * n))
65         self.J[:n, n:] = np.eye(n)
66         self.J[n:, :n] = -np.eye(n)

```

呼び出し側 (make_static_parents_N3_N40_v1.py、式 9) :

```

48         sys_lr = LowRankSystem(N)
49         rng = np.random.default_rng(40260722 + 1000 * N + SEED)

```

■4.3.2 ループ処理 (1) : 自己無撞着親の構成 (式 2~式 6) 生成子の構成 (式 2・式 3。set_theta は c, s と解析的 G を保持する) :

```

68     def set_theta(self, theta):
69         n = self.n
70         self.c = np.cos(theta)
71         self.s = np.sin(theta)
72         T = np.zeros((n, n))
73         T[self.ea, self.eb] = theta
74         T[self.eb, self.ea] = theta
75         CT = np.cos(T)
76         ST = np.sin(T)
77         np.fill_diagonal(CT, 0.0)
78         np.fill_diagonal(ST, 0.0)
79         Gcc = CT * CT
80         Gcs = CT * ST
81         Gss = ST * ST
82         np.fill_diagonal(Gcc, Gcc.sum(axis=1))
83         np.fill_diagonal(Gcs, Gcs.sum(axis=1))
84         np.fill_diagonal(Gss, Gss.sum(axis=1))
85         G = np.empty((2 * n, 2 * n))
86         G[:n, :n] = Gcc
87         G[:n, n:] = Gcs
88         G[n:, :n] = Gcs
89         G[n:, n:] = Gss
90         self.G = G

```

親反復本体 (式 4: 169-172 行、式 5: 173-175 行、式 6 の判定: 176-181 行) :

```

158 def make_parent(sys_lr, rng, iters=400, beta=0.5, tol=1e-8, restarts=3):
159     """自己無撞着円偏波固有モード親。小空間 (JG) の固有対で反復。
160
161     収束判定 (残差 < tol) 付き。停滞時はランダム初期位相からリスタート。

```

```

162     """
163     best = (None, np.inf, None)
164     for _ in range(restarts):
165         theta = rng.uniform(0.0, 2.0 * np.pi, sys_lr.m)
166         v = None
167         for it in range(iters):
168             sys_lr.set_theta(theta)
169             ev, EV = np.linalg.eig(sys_lr.J @ sys_lr.G)
170             idx = int(np.argmin(ev.imag)) #  $\lambda = -i \sigma_{max}$ 
171             v = sys_lr.w(EV[:, idx].astype(complex))
172             v = v / np.linalg.norm(v)
173             theta_new = np.angle(v)
174             mix = (1.0 - beta) * np.exp(1j * theta) + beta * np.exp(1j * theta_new)
175             theta = np.angle(mix)
176             if it % 10 == 9:
177                 sys_lr.set_theta(np.angle(v))
178                 res_now = _eigenmode_residual(sys_lr, v)
179                 progress(f"親構成 iter={it+1} 残差={res_now:.2e}")
180                 if res_now < tol:
181                     break
182             sys_lr.set_theta(np.angle(v))
183             residual = _eigenmode_residual(sys_lr, v)
184             if residual < best[1]:
185                 best = (v, residual, sys_lr.sigma_spectrum())
186             if residual < tol:
187                 break
188         v, residual, sig = best
189         sys_lr.set_theta(np.angle(v))
190     return v, residual, sig

```

残差 (式 6) :

```

151 def _eigenmode_residual(sys_lr, v):
152     """カイラリティ非依存の固有モード残差:  $\mu = v^\dagger (iKv)$  に対する  $\| iKv - \mu v \|$ 。"""
153     kv = sys_lr.kmatvec(v)
154     mu = float(np.real(np.conj(v) @ (1j * kv)))
155     return float(np.linalg.norm(1j * kv - mu * v))

```

停止条件・例外的挙動 (数式化しない要素) : - 収束判定は 10 反復ごと (176 行 `it % 10 == 9`) にのみ行われる。したがって実際の 反復回数は 10 の倍数で終わる。- 上限 `iters=1200` に達しても `r < tol` にならない場合は例外を投げず、リスタート (165 行、`rng` から新しい初期位相を消費) へ進む。最大 `restarts=3`。最良残差の解が採用される (163, 184-185 行)。リスタートが起きると `rng` 消費数が変わるため、再現時は同一 `tol` の使用が必須である (本走行では全 `N` が第 1 リスタート内で 収束し、追加消費は発生していない。根拠: `N=40` の生成物が、同条件の正本と bit 一致)。- `progress` 行 (179 行) は `stderr` 出力のみで演算に影響しない。

■4.3.3 ループ処理 (2) : 零閉塞核種の構成 (式 7)

```

193 def zero_closure_kernel_seed(sys_lr, rng):
194     """span(W) の直交補内の実正規直交対 (u,w) から  $g=(u+iw)/\sqrt{2}$ 。  $g^T g = 0$  厳密。"""
195     m = sys_lr.m
196     def project_out(g):
197         q = np.linalg.lstsq(sys_lr.G, sys_lr.wt(g), rcond=None)[0]
198         return g - sys_lr.w(q)
199     u = project_out(rng.normal(size=m))
200     u = u / np.linalg.norm(u)
201     w = project_out(rng.normal(size=m))
202     w = w - (w @ u) * u
203     w = w / np.linalg.norm(w)
204     return (u + 1j * w) / math.sqrt(2.0)

```

- 射影 (式 7 の $P \perp$) は正規方程式 $G q = W^T g$ を `lstsq (rcond=None)` で解く (197 行)。G が特異に近い場合も `lstsq` は最小二乗解を返すため例外は生じない。
- この時点の G は、`make_parent` 終了時に設定された $\theta = \arg v$ (189 行) のものである。すなわち種は「親の生成子の値域の直交補」から取られる。

■4.3.4 終了処理: Z0 構成・ゲート・保存・台帳 `make_static_parents_N3_N40_v1.py` (式 8: 52-53 行) :

```

50     v, residual, sig = make_parent(sys_lr, rng, iters=ITERS, tol=TOL)
51     g = zero_closure_kernel_seed(sys_lr, rng)
52     Z = v + DELTA * g
53     Z = Z / np.linalg.norm(Z)
54     converged = bool(residual < 1e-8)
55     status = "ok" if converged else "NOT_CONVERGED"
56
57     if N == 40:
58         ref = np.load(REF40)
59         same = all(np.array_equal(x, ref[k]) for x, k in ((v, 'v'), (g, 'g'), (Z, 'Z0')))
60         print(f"GATE N=40 v/g/Z0 bit-identical to canonical static parent: {same}")
61         if not same:
62             gate_ok = False
63             status = "GATE_FAIL"
64
65     if os.path.exists(out_path):
66         prev = np.load(out_path)
67         same_prev = all(np.array_equal(x, prev[k]) for x, k in ((v, 'v'), (g, 'g'), (Z,
68         print(f"N={N}: 既存ファイルあり (上書きせず検証のみ) 一致={same_prev} residual={residual}")
69         if not same_prev:
70             gate_ok = False
71             status = "EXISTING_MISMATCH"

```

```

72         else:
73             np.savez_compressed(out_path, v=v, g=g, Z0=Z,
74                                 sigma=sig, residual=np.float64(residual),
75                                 n=np.int64(N), seed=np.int64(SEED),
76                                 delta=np.float64(Delta), tol=np.float64(TOL),
77                                 iters=np.int64(ITERs))

81     with open(os.path.join(PARENT_DIR, "parents_summary.csv"), "w", newline="") as fh:
82         w = csv.writer(fh)
83         w.writerow(["N", "M", "parent_residual", "rank_planes", "status"])
84         w.writerows(rows)
85     n_bad = sum(1 for r in rows if r[4] != "ok")
86     print(f"summary: {len(rows)} parents, {n_bad} non-ok")
87     if not gate_ok:
88         print("GATE FAIL")
89         sys.exit(1)
90     print("STATIC PARENTS DONE")

```

例外的挙動: - 既存ファイルは上書きしない。既存がある場合は再生成値との bit 一致検証のみ行い (65-71 行)、不一致は EXISTING_MISMATCH としてゲート不合格にする。- 収束不良 ($\text{residual} \geq 1e-8$) は例外ではなく NOT_CONVERGED として台帳に明示される (54-55 行)。- いずれかのゲート不合格で終了コード 1 (87-89 行)。

5. 実行結果

5.1 再現コマンド

```

cd N3_N40_stage123_sweep_20260905
python3 make_static_parents_N3_N40_v1.py      # 単独実行
# または ./run_all.sh の第 1 段として実行される

```

5.2 実行環境

- Python 3.9.6 (`.venv/bin/python3`)
- numpy 2.0.2 (BLAS/LAPACK: macOS Accelerate)
- macOS 26.3.1 (arm64, Darwin 25.x)
- 乱数: numpy default_rng (PCG64)

5.3 実行時間

全 38 親 (生成・保存・検証・台帳出力込み) でおおよそ 30 秒~1 分。1 親あたりの支配項は JG ($2N \times 2N$) の固有分解 $\times$ 反復数で、 $N=40$ は約 0.15 秒/親構成。

5.4 検証ゲート

ゲート	合格条件	実測	合否
G1: N=40 系譜	生成した $v \cdot g \cdot Z0$ が正本静的親 (20260904、7 月正本走行と bit 一致検証済み) と <code>np.array_equal</code> で一致	一致	PASS
G1': ファイル同一性	(観察) N=40 npz の SHA256 が正本静的親ファイルと一致	両者 <code>eadc87ee...</code> で同一	PASS
G2: 収束	全 N で <code>residual < 1e-8</code> (<code>status=ok</code>)	38/38 ok、 <code>residual ∈ [3.54e-14, 9.42e-13]</code>	PASS
G3: 実行	終了コード 0・STATIC PARENTS DONE 出力	確認	PASS

5.5 データ

項目	内容
フォルダ 静的親 npz	N3_N40_stage123_sweep_20260905/parents/ parent_static_N00003..N00040_makeparent_20260905.npz (38 個)
サイズ	約 2.0KB (N=3、1,980 bytes) ~ 約 37KB (N=40、 37,396 bytes)、合計約 636KB
台帳	parents_summary.csv (38 行: N, M, parent_residual, rank_planes, status)
SHA256	全ファイルの正本値は同梱 SHA256SUMS.txt に収録。 代表値:

```

c3230103e82976decde7bbe6fe5df545d12804127cf274f0d385e044ce544ca2  make_static_parents_N3_N40_v1.py
ba0fc19b03caf06d16e97e2cd5da499ed2ed8e95288f6dbf2062bedcaf11176d  run_n_scaling_lowrank_v1.py
eadc87ee0276554c7ab02e571e05200f0b719c1250b82607c7546b30a4d6f232  parents/parent_static_N00040_makeparent_20260905.npz
f384d9e90cc0a3a87a82a9a6928c9996d51bd0f83862c79f2cfe8a5ce182bbca  parents/parents_summary.csv

```

5.6 図化

本章では図を生成しない(初期データの複素平面図は第 4 章の step0 グリッド図 `fig_complex_plane_step0_N3_N40_stage123` が本章の生成物を可視化する)。

6. 実行分析 (客観的報告と観察のみ)

- 全 38 親が第 1 リスタート内で収束し、固有モード残差は $3.54 \times 10^{-14} \sim 9.42 \times 10^{-13}$ の範囲に収まった (`tol=1e-12` に対し全て同桁以下)。NOT_CONVERGED・GATE_FAIL は 0 件。
- N=40 の生成物は、独立に (2026-09-04 に別フォルダで) 生成された正本静的親と配列レベルで bit 一致し、npz ファイル自体の SHA256 も一致した。同一 seed・同一エンジン・同一環境の下で、生成

が完全に決定的であることの直接の証拠である。

3. 親の σ スペクトルの非零平面数 (rank_planes) は $N=3$: 2、 $N=4$: 2、 $N=5$: 4、 $N=6$: 6、 $N \geq 7$: N と観測された (台帳 `parents_summary.csv` 参照)。
4. ファイルサイズは $M = N(N - 1)/2$ にほぼ比例して増加した。
5. 乱数消費は「一様 m 個 + 正規 $2m$ 個」(リスタートなしの場合) で、全走行がこの最小消費で完了している (項目 2 の bit 一致がその傍証)。

(第 1 章おわり。第 2 章「段 1+2+3 力学の定義と由来」に続く)